What we did

RL post-training over #163's multi-draft contacts-v1 model, which writes ~15 near-disjoint candidate contact maps in one generation. The reward is dense: every emitted `<contact> <pI> <pJ>` triple is scored individually against ground truth, rather than one scalar being smeared over ~4,200 tokens.

Two terms, both landing per token. A stepwise term paying +(1-p̄) for a true contact and -p̄·δ^e for a false one, where e counts earlier errors in the section and p̄ is the policy's own recent precision. And a document-level term, the best section's F1, baselined leave-one-out across the generations for a protein.

Fully online marin.rl: vLLM rollout workers and a levanter trainer with live weight transfer, on marin iris v5p.

Why $\bar{p}$ is the policy's own precision

This is the single most important constant in the design. Per-contact precision is only about 0.23, so a FIXED penalty for a wrong contact makes "emit nothing" the optimal policy, and the run collapses to empty sections.

Centring the reward on $\bar{p}$ — an EMA of the policy's own recent precision — makes E[stepwise reward] about zero at current performance, so the gradient says only "beat yourself". Measured through training: `train/mean_advantages` = 0.0028.

Result: the reward worked, the headline did not move

Scored on all 554 eval proteins x 4 rollouts, paired per protein, by the same code that produced the arm-F reference.

Per-contact precision +0.0085 (+4.6σ). First-candidate F1 +0.0128 (+5.1σ). Candidates got better, which is what a per-contact reward targets.

best-of-N F1: +0.0008 (+0.4σ). The primary criterion was +0.02 at 3σ. NOT MET.

Why: best-of-N is quality times spread

Candidates per generation fell 1.35 (-7.5σ) and inter-candidate Jaccard rose 0.0087 (+7.7σ, on 72.6% of proteins). The run traded quality for spread almost exactly evenly.

This is the tension the design anticipated: the stepwise term pushes every section toward the model's single best guess, and the document-level term exists to pay for spread. At λ_step = λ_doc = 1.0, the stepwise term won.

No reward hacking: contacts per section were unchanged (92.7 -> 92.8), so the drop in total predictions is fewer candidates, not shorter ones.

The next lever is the λ ratio, not the learning rate

Raise λ_doc relative to λ_step, or reward spread explicitly rather than only the best section's F1. Worth combining with a higher learning rate — KL of 0.00051 says this policy barely moved — but a bigger LR alone would likely buy a larger version of the same trade.

Follow-up filed as #208: the same dense reward on the base `<contacts-v1>` format only, where the spread axis disappears, with the document term redefined as a rollout's leave-one-out marginal contribution to the n=100 consensus vote — which is the metric actually reported in model summaries.

Infrastructure, and what it ran on

Per arm: 4 x v5p-8 rollout generators (vLLM, tp=4) plus 1 x v5p-8 trainer, with Arrow Flight moving 2.9 GB of bf16 weights every 8 steps, and 2 CPU pods for the coordinator and driver. Offline: pool build on CPU, checkpoint export on CPU, evaluation on 1 x v5p-8.

Four rollout workers took `rollout_wait_duration` from 36.1 s to 0.0 — the trainer was starved, not transfer-bound, and an earlier reading of that as a weight-transfer cost was wrong. v5p-16 had 0 ready slices against 121 v5p-8, so the trainer was sized to what schedules, dropping batch 128 -> 32.

What we trust, and what bit

The sampling path reproduces #163's published numbers: best 0.3015 against 0.3025 over 2,216 rollouts, with two independent scorers — a token-id walk and a text regex — agreeing to 0.0.

Five bring-up failures, all caught by a 10-step gate rather than a three-arm sweep, and none reachable from the parity gate: marin deleting `marin.rl` mid-project, an unpropagated W&B key, a `canonical_model_name` that is both a substring match and an exact key, a `prng_key` union type, and a per-step weight transfer costing 372 s.

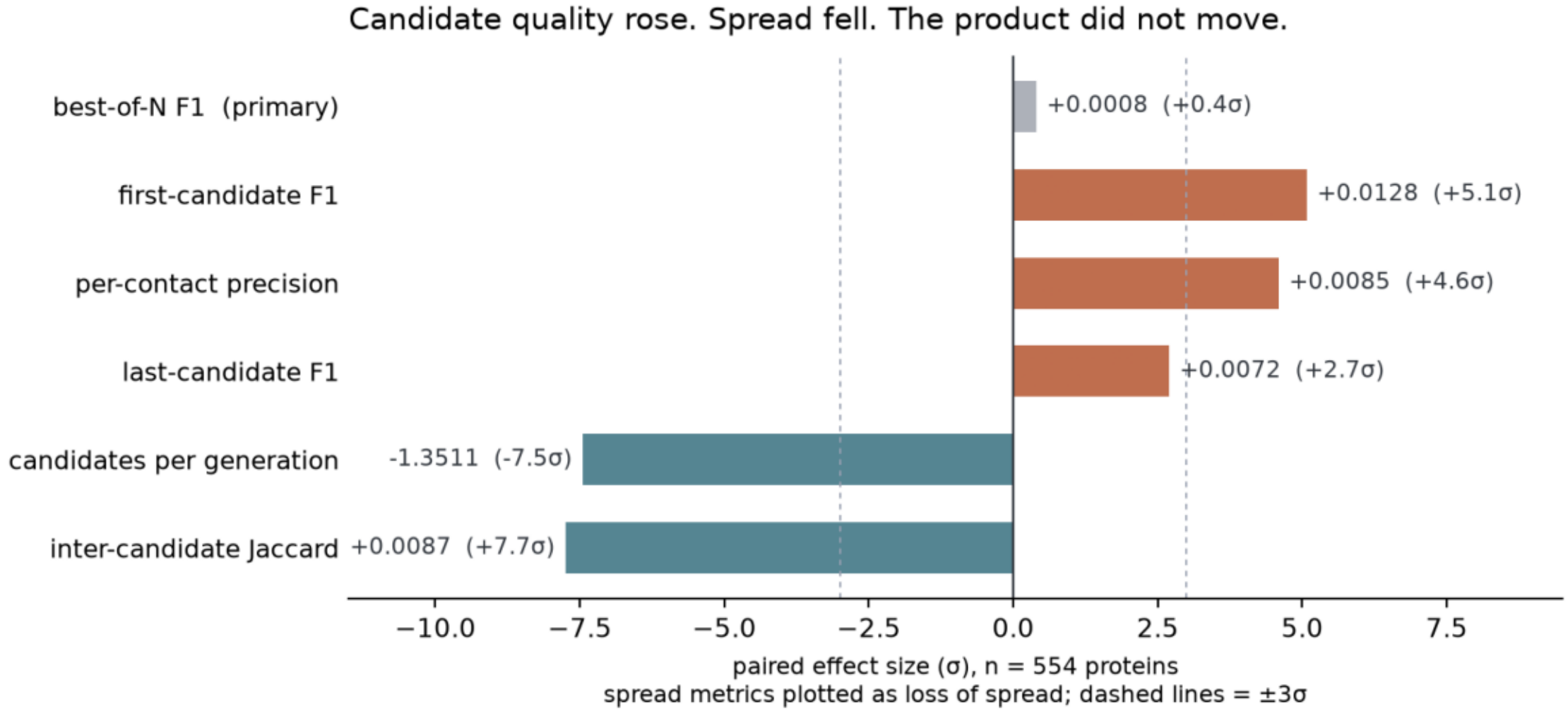
Three of our own guards failed toward false reassurance — a test fake more permissive than the real tokenizer, a reaper counting a crashed arm as finished, another reading cached values as current. Each answered "is this fine?" from information it could not see.

Caveats

Only 1 of 3 learning-rate arms survived preemption, so this is a single point rather than a sweep. The base-task guardrail, teacher-forced R-precision in plain mode, is UNMEASURED, so that kill criterion is unverified.

effect_sizes.png

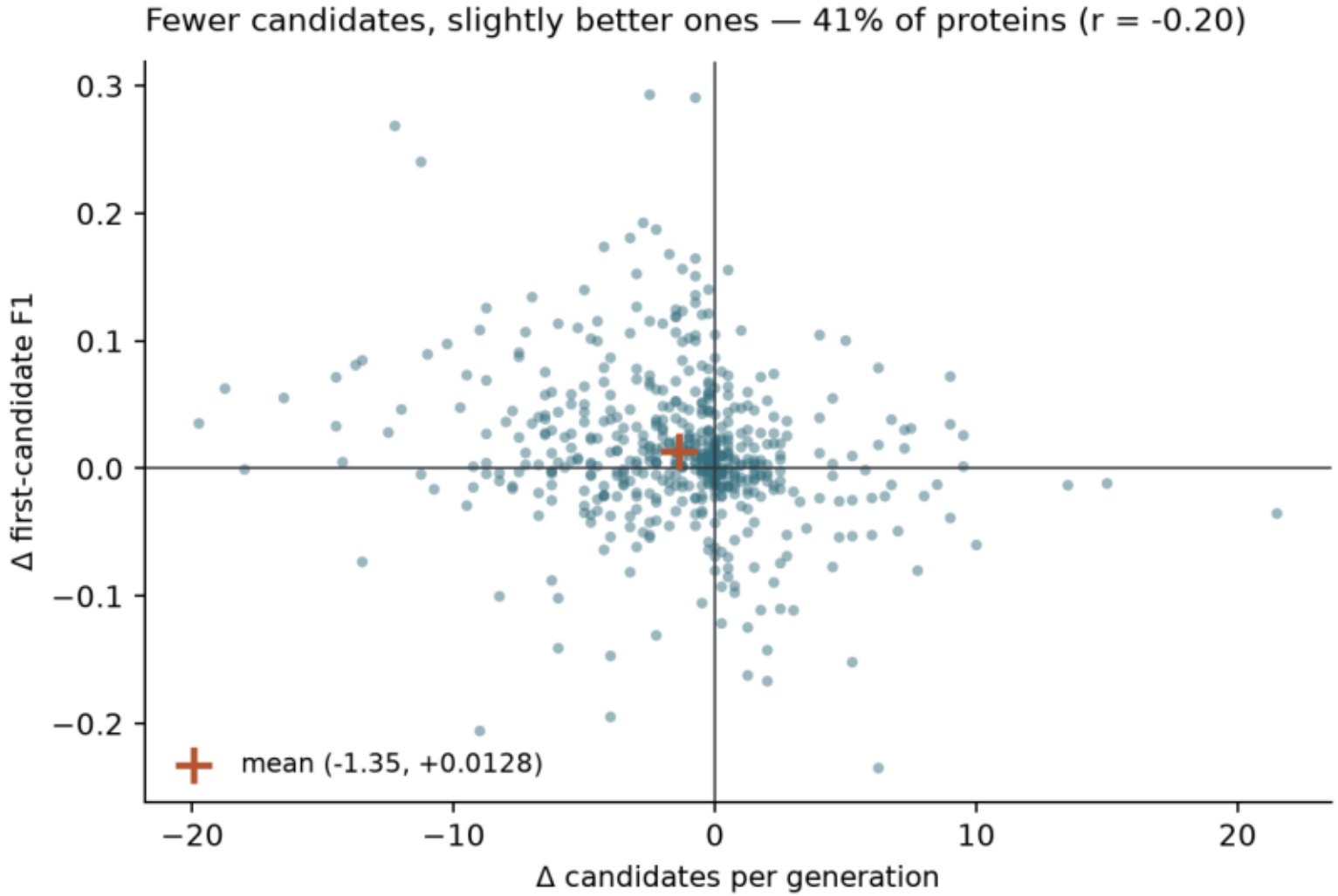
Paired per-protein effect sizes, 1e-6 arm vs #163 arm F. Quality rose at 4.6-5.1 sigma, spread fell at 7.5-7.7; their product, best-of-N F1, sits at +0.0008.



$ plot_results.py

quality_vs_spread.png

Per protein: change in first-candidate F1 vs candidate count. Better-and-fewer is the plurality quadrant at 41%, and the trade is weak per protein (r = -0.20) even though both means shift.



$ plot_results.py